

Praktikum Pemrograman Berorientasi Objek

Pertemuan 5

Materi Pertemuan 5

Gambaran Materi Pembelajaran Hari Ini

Materi Pertemuan 5

01

UML

Memahami perancangan sistem berorientasi objek menggunakan
Unified Modeling Language

02

Recalling

Mengingat kembali dan mengimplementasikan materi PBO yang
telah dipelajari

UML (Unified Modeling Language)

Definisi

UML adalah salah satu "bahasa" standar yang digunakan untuk memodelkan visualisasi, perancangan, dan pendokumentasian aplikasi.

"Bahasa" di sini bukanlah bahasa pemrograman, melainkan berupa **simbol grafis, bangun datar, dan garis koneksi** yang standar.

Analogi Blueprint

UML ibarat sebuah *blueprint* (cetak biru) untuk membangun gedung.

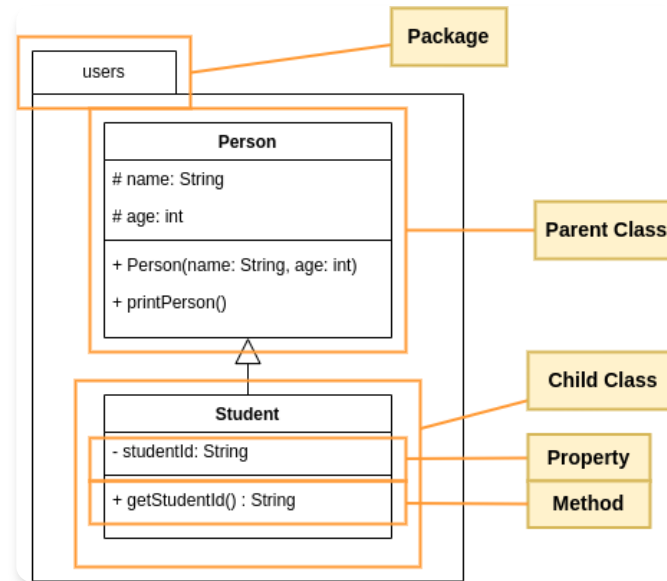
Bangunan yang kompleks tentu membutuhkan *blueprint* yang berbeda-beda tergantung fungsinya (misal: denah per lantai, skematik jalur pipa, atau rute evakuasi). Inilah alasan mengapa UML memiliki **banyak tipe diagram** sesuai tujuannya.

Tipe-tipe Diagram UML:

- **Class diagram** (dibahas di sini)
- Use case diagram
- Composite structure diagram
- State machine diagram
- Sequence diagram
- Collaboration diagram
- Deployment diagram
- *dan lain-lain...*

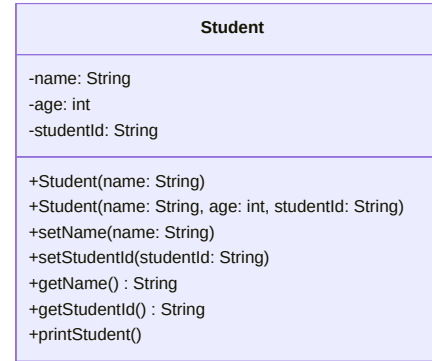
Class Diagram

- Salah satu tipe UML yang sering digunakan adalah class diagram.
- Class diagram menunjukkan struktur dari suatu sistem, seperti kelas, method, atribut, dan hubungan di antaranya.
- Class diagram membantu kita untuk memahami bagaimana sistem diatur dan bagaimana interaksi antar komponen terjadi.



Contoh kelas `Student` dan bagaimana menggambarannya di diagram:

```
class Student {  
    private String name;  
    private int age;  
    private String studentId;  
  
    public Student(String name) { ... }  
    public Student(String name, int age, String studentId) { ... }  
  
    public void setName(String name) { ... }  
    public void setStudentId(String studentId) { ... }  
  
    public String getName() { ... }  
    public void printStudent() { ... }  
}
```



Visibilitas Class Diagram

Cara menentukan access modifier/visibilitas dari atribut dan method pada class diagram UML:

Simbol	Access Modifier
+	public
-	private
#	protected
~	package

Jenis Hubungan Antar Kelas

Relasi	Makna	Ciri utama	Simbol UML
Aggregation	<i>has-a</i> (lemah)	Bagian bisa tetap hidup tanpa keseluruhannya	Garis dengan belah ketupat kosong
Composition	<i>has-a</i> (kuat)	Bagian ikut hilang bersama keseluruhannya	Garis dengan belah ketupat terisi
Inheritance	<i>is-a</i>	Subclass mewarisi atribut dan method dari superclass (<code>extends</code>)	Garis dengan segitiga kosong
Realization	<i>implements</i>	Class memenuhi kontrak sebuah interface (<code>implements</code>)	Garis putus-putus dengan segitiga kosong

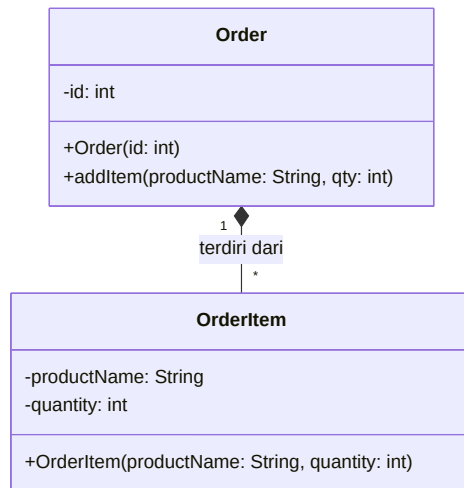
Aggregation

Hubungan has-a di mana bagian dibuat di luar lalu hanya *dimasukkan* ke keseluruhan. Bagian tetap ada walau keseluruhannya hilang.

Composition

Hubungan has-a yang lebih kuat: bagian dibuat dan dimiliki *di dalam* keseluruhan, sehingga siklus hidupnya mengikuti pemiliknya.

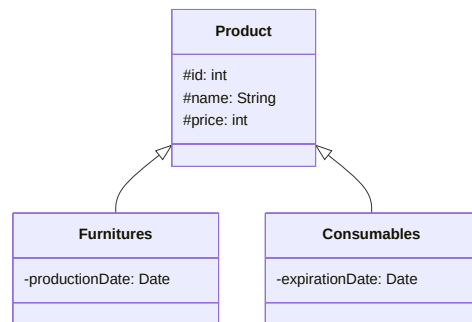
```
class OrderItem {  
    private String productName;  
    private int quantity;  
    public OrderItem(String productName, int quantity) { ... }  
}  
  
class Order {  
    private int id;  
    private List<OrderItem> items = new ArrayList<>();  
  
    public Order(int id) { this.id = id; }  
  
    public void addItem(String productName, int qty) {  
        // OrderItem dibuat DI DALAM Order  
        items.add(new OrderItem(productName, qty));  
    }  
}
```



Inheritance

Hubungan is-a: *subclass* mewarisi atribut dan method dari *superclass*, lalu boleh menambahkan miliknya sendiri.

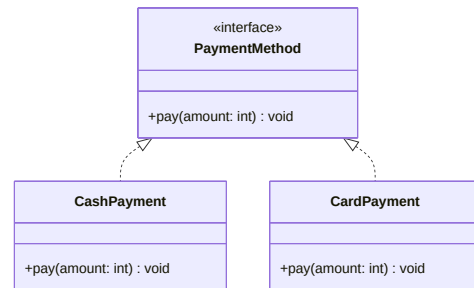
```
class Product {  
    protected int id;  
    protected String name;  
    protected int price;  
    public Product(int id, String name, int price) { ... }  
}  
  
class Furnitures extends Product {  
    private Date productionDate;  
  
    public Furnitures(int id, String name, int price,  
                      Date productionDate) {  
        super(id, name, price); // constructor Product  
        this.productionDate = productionDate;  
    }  
}
```



Realization

Class memenuhi kontrak sebuah interface: interface hanya menjanjikan *apa* yang bisa dilakukan, class yang menentukan *bagaimana* caranya.

```
interface PaymentMethod {  
    void pay(int amount);  
}  
  
class CashPayment implements PaymentMethod {  
    @Override  
    public void pay(int amount) {  
        System.out.println("Bayar tunai: " + amount);  
    }  
}  
  
class CardPayment implements PaymentMethod {  
    @Override  
    public void pay(int amount) {  
        System.out.println("Bayar kartu: " + amount);  
    }  
}
```



Recalling Materi

Review Konsep & Sintaks PBO (Minggu 1 - 4)

Recalling: Struktur Class & Encapsulation

Class & Object: Class adalah blueprint, object adalah realisasinya (via `new`). Atribut/properti mendefinisikan *state*, method mendefinisikan *behavior*.

Encapsulation: Praktik menyembunyikan detail internal class (Data Hiding) lewat *access modifier* `private`. Interaksi dari luar hanya diizinkan via `public` method (Getter/Setter).

```
public class User {  
    // Properti (private untuk Encapsulation)  
    private String username;  
    private String password;  
  
    // ... Constructor & Method ...  
}  
  
// Instansiasi Object  
User user1 = new User();
```

Recalling: Constructor & Getter-Setter

Constructor: Dipanggil otomatis saat objek dibuat. Digunakan untuk inisialisasi awal.

Getter & Setter: Method standar untuk mengambil nilai (`get`) dan mengubah nilai (`set`) atribut *private* secara aman (*Data Integrity*).

```
public class User {  
    private String password;  
  
    // Constructor  
    public User(String password) {  
        this.password = password; // 'this' merujuk ke atribut  
    }  
  
    // Setter (Bisa ditambahkan validasi)  
    public void setPassword(String password) {  
        if(password.length() > 5) this.password = password;  
    }  
  
    // Getter  
    public String getPassword() {  
        return this.password;  
    }  
}
```

Recalling: Inheritance & Overriding

Inheritance (`extends`): *Subclass* (anak) mewarisi *Superclass* (induk).

`super` : Keyword pemanggil konstruktor atau method milik induk.

Overriding (`@Override`): *Subclass* menimpa paksa logika method bawaan dari parent-nya.

```
class Hewan {  
    public Hewan() { System.out.println("Hewan dibuat"); }  
    public void bersuara() { System.out.println("..."); }  
}  
  
class Kucing extends Hewan { // Inheritance  
    public Kucing() {  
        super(); // Memanggil constructor parent  
    }  
  
    @Override // Menimpa method parent  
    public void bersuara() {  
        System.out.println("Meow!");  
    }  
}
```

Recalling: Kata Kunci this & super

Kata Kunci `this` : Berfungsi untuk merepresentasikan (menunjuk) atribut atau *method* milik instansiasi objek dari kelas itu sendiri. Biasanya digunakan untuk membedakan antara variabel milik kelas dengan *parameter* lokal.

Kata Kunci `super` : Berfungsi untuk merepresentasikan (menunjuk) atribut atau *method* dari *parent class*. Pada *constructor subclass*, `super()` digunakan untuk memanggil dan memberikan data ke *constructor parent*.

```
public class Character {
    protected String name;

    public Character(String name) {
        // 'this' membedakan atribut dengan parameter lokal
        this.name = name;
    }
}

public class Warrior extends Character {
    private double hp;

    public Warrior(String name, double hp) {
        // 'super' mengirim name ke constructor parent
        super(name);
        this.hp = hp;
    }

    public void printStats() {
        // 'super' juga bisa panggil method/atribut parent
        System.out.println("Nama: " + super.name);
        System.out.println("HP : " + this.hp);
    }
}
```

Recalling: Method Overloading

Method Overloading: Mekanisme menulis ulang method dengan **nama yang sama**, tetapi **berbeda parameter** (jumlah, tipe data, atau urutan). Implementasinya bisa berbeda.

Compiler akan otomatis menentukan fungsi mana yang paling cocok dengan inputan.

(Catatan: TIDAK BOLEH overload hanya berdasarkan perbedaan return type)

```
public class Kalkulator {  
  
    // Method 1: Dua parameter int  
    public int tambah(int a, int b) {  
        return a + b;  
    }  
  
    // Method 2: Tiga parameter int (Overloaded)  
    public int tambah(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Method 3: Parameter double (Overloaded)  
    public double tambah(double a, double b) {  
        return a + b;  
    }  
}
```

Recalling: Abstraksi (Abstract & Interface)

Abstract Class: Kelas yang tak dapat diinstansiasi langsung via `new`. *Abstract Method* dideklarasikan tanpa *body/implementasi*. *Subclass* wajib meng-*override*-nya.

Interface (implements): Berbentuk abstrak implisit. Ibarat sebuah **Contract** yang wajib dipatuhi. Semua method otomatis memiliki akses modifier `public`.

```
// Interface
interface BisaTerbang {
    void terbang(); // otomatis public abstract
}

// Abstract Class
abstract class Burung {
    public abstract void bersuara(); // abstrak, no body
    public void makan() { System.out.println("Makan"); }
}

// Implementasi
class Elang extends Burung implements BisaTerbang {
    @Override public void bersuara() { System.out.println("Kwakk"); }
    @Override public void terbang() { System.out.println("Terbang") }
}
```

Recalling: Polimorfisme

Polimorfisme: Kemampuan untuk mempunyai beberapa bentuk class yang berbeda. Terjadi saat suatu objek bertipe **parent class**, tetapi **pemanggilan constructornya melalui child class**.

Contohnya `Kendaraan bmw = new Mobil();` Satu referensi array parent (`Kendaraan[]`) dapat menampung campuran berbagai objek *child* nyata.

```
public class Main {  
    public static void main(String[] args) {  
  
        // Referensinya 'Burung' (Parent), Objeknya 'Elang' (Child)  
        Burung burungSatu = new Elang();  
  
        // Referensinya 'Burung' (Parent), Objeknya 'Bebek' (Child)  
        Burung burungDua = new Bebek();  
  
        // Array Polimorfisme  
        Burung[] koleksi = { burungSatu, burungDua };  
  
        for(Burung b : koleksi) {  
            b.bersuara(); // Panggilan polimorfik  
        }  
    }  
}
```

Tugas Minggu 5

Kerjakan tugas sebagai bagian dari latihan!

Instruksi Tugas

- Clone repo tugas dari Classmoji masing-masing.
- Buka dokumen pada link di sini.
- Baca soal dengan seksama. Setiap soal sudah ditambahkan unit test. Jalankan `mvn test` atau `mvn test -Dtest.groups=problem1` (seperti yang ada di dokumen) untuk menguji kebenaran program.
- Tugas diselesaikan hari ini sebagai sarana latihan.

Terima Kasih!

Tertekan ya? Namanya juga hidup